

Module 10: Cross-Program Invocations (CPIs)

Program Composition & Interoperability

What are Cross-Program Invocations?

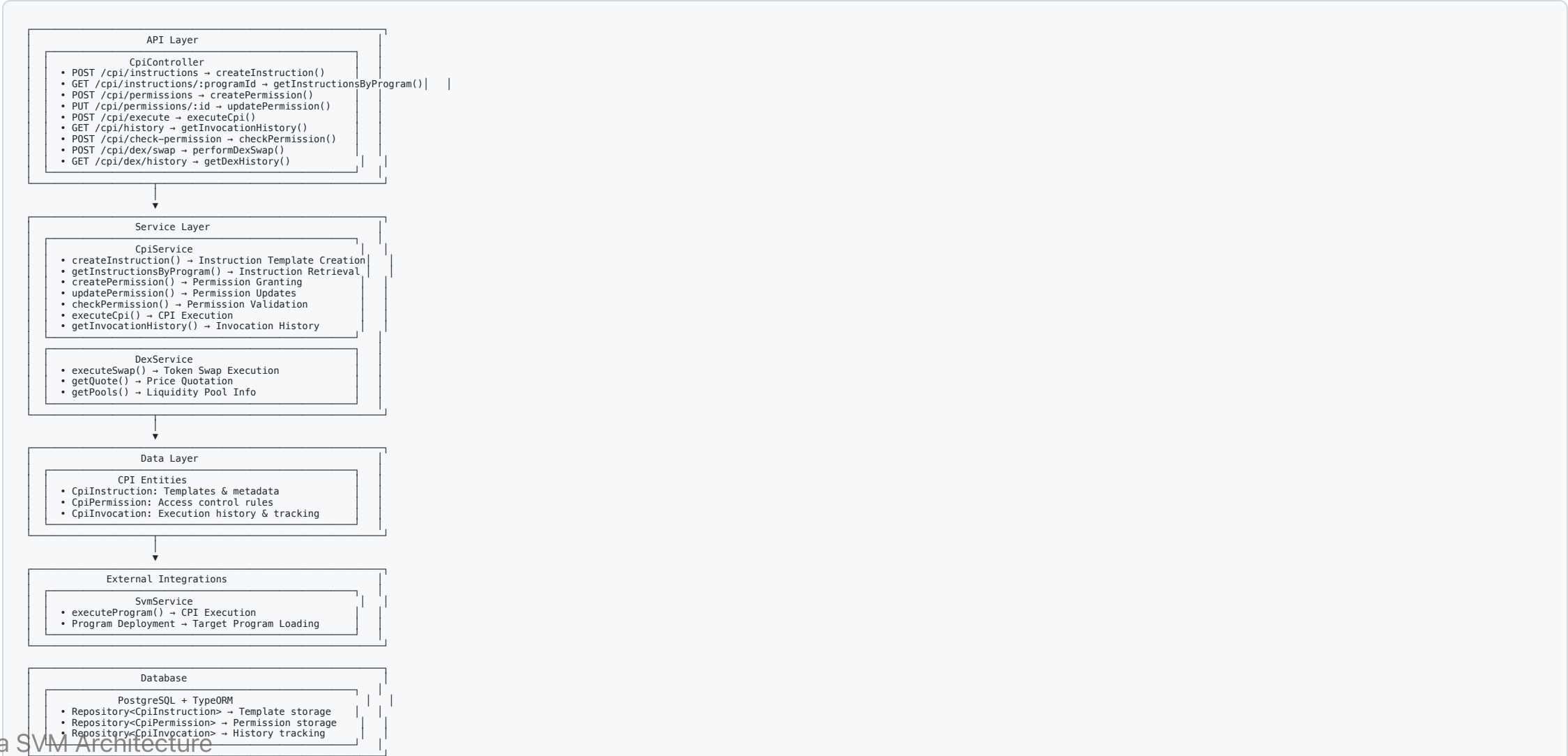
CPI Fundamentals

- **Program Calls:** One program invoking another program's instructions
- **Composability:** Building complex functionality from simple programs
- **Trustless Execution:** Programs execute with full SVM runtime security
- **Atomic Operations:** All CPI calls succeed or fail together

SVM Advantages

- **Single Transaction:** Multiple program interactions in one atomic operation
- **Shared State:** Programs can read and modify shared accounts
- **Gas Efficiency:** No additional transaction overhead for CPI calls
- **Security Isolation:** Each program maintains its own security boundaries

Architecture Overview



CPI Permission System

Permission Management

```
interface CpiPermission {  
  id: string;  
  programId: string;           // Target program being granted access  
  granterProgramId: string;    // Program granting the permission  
  permissionType: 'read' | 'write' | 'execute';  
  accountId?: string;          // Specific account (optional)  
  expiresAt?: Date;            // Permission expiration  
  isActive: boolean;  
}
```

Permission Validation

```
async checkPermission(  
  callerProgramId: string,  
  targetProgramId: string,  
  permissionType: string,  
  ...args: any[]  
) {  
  // Validation logic  
}
```

Instruction Templates

CPI Instruction Structure

```
interface CpiInstruction {  
    id: string;  
    programId: string;           // Target program ID  
    callerProgramId: string;     // Calling program ID  
    instructionData: any;        // Instruction payload  
    accounts: AccountMeta[];     // Account metadata array  
    methodName: string;          // Method identifier  
    requiresPermission: boolean; // Permission check required  
    permissionLevel: string;     // Required permission type  
    isActive: boolean;  
}  
  
interface AccountMeta {  
    pubkey: PublicKey;  
    isSigner: boolean;  
    isWritable: boolean;  
}
```

CPI Execution Flow

Execution Process

```

async executeCpi(executionDto: ExecuteCpiDto): Promise<CpiInvocation> {
  // 1. Permission Check
  if (executionDto.requiresPermission) {
    const hasPermission = await this.checkPermission(
      executionDto.callerProgramId,
      executionDto.targetProgramId,
      executionDto.permissionType
    );

    if (!hasPermission) {
      throw new ForbiddenException('Insufficient CPI permissions');
    }
  }

  // 2. Create invocation record
  const invocation = await this.createInvocationRecord(executionDto);

  try {
    // 3. Execute via SVM service
    const result = await this.svmService.executeProgram({
      programId: executionDto.targetProgramId,
      instruction: executionDto.instruction,
      accounts: executionDto.accounts
    });

    // 4. Update invocation with success
    invocation.status = 'success';
    invocation.transactionId = result.signature;
    invocation.gasUsed = result.computeUnits;
  } catch (error) {
    // 5. Record failure
    invocation.status = 'failed';
    invocation.errorMessage = error.message;
  }
}

```

DEX Operations

Token Swap Implementation

```

async performDexSwap(swapDto: DexSwapDto): Promise<SwapResult> {
  // Get quote from DEX
  const quote = await this.getQuote({
    inputMint: swapDto.inputMint,
    outputMint: swapDto.outputMint,
    amount: swapDto.amount,
    slippage: swapDto.slippage
  });

  // Check slippage tolerance
  if (quote.priceImpact > swapDto.maxSlippage) {
    throw new BadRequestException('Price impact too high');
  }

  // Execute swap via CPI
  const swapInstruction = await this.createSwapInstruction(quote);

  const result = await this.cpiService.executeCpi({
    callerProgramId: this.dexProgramId,
    targetProgramId: quote.ammProgramId,
    instruction: swapInstruction,
    accounts: quote.accounts,
    requiresPermission: true,
    permissionType: 'execute'
  });

  return {
    signature: result.transactionId,
    outputAmount: quote.outputAmount,
    priceImpact: quote.priceImpact
  };
}

```

Common CPI Patterns

Token Operations

```
// SPL Token Transfer via CPI
const tokenTransferIx = Token.createTransferInstruction(
  TOKEN_PROGRAM_ID,
  sourceTokenAccount,
  destinationTokenAccount,
  owner,
  [],
  amount
);

// Execute via CPI
await cpiService.executeCpi({
  callerProgramId: myProgramId,
  targetProgramId: TOKEN_PROGRAM_ID,
  instruction: tokenTransferIx,
  accounts: [
    { pubkey: sourceTokenAccount, isSigner: false, isWritable: true },
    { pubkey: destinationTokenAccount, isSigner: false, isWritable: true },
    { pubkey: owner, isSigner: true, isWritable: false }
  ]
});
```

Security Features

CPI Security Measures

- **Permission Validation:** Granular access control enforcement
- **Account Verification:** Address validation and ownership checks
- **Execution Isolation:** Failure containment within CPI boundaries
- **Audit Trail:** Complete invocation logging and tracking
- **Gas Metering:** Resource usage monitoring and limits

Error Handling

```
class CpiExecutionError extends Error {  
  constructor(  
    public readonly invocationId: string,  
    public readonly errorCode: string,  
    public readonly details: any  
  ) {}  
}
```

API Endpoints

Instruction Management

- `POST /cpi/instructions` - Create CPI instruction template
- `GET /cpi/instructions/:programId` - Get instructions for program

Permission Management

- `POST /cpi/permissions` - Create CPI permission
- `PUT /cpi/permissions/:id` - Update permission
- `POST /cpi/check-permission` - Validate permission

Execution & History

- `POST /cpi/execute` - Execute CPI call
- `GET /cpi/history` - Get invocation history

Advanced CPI Patterns

Program Derived Address Signing

```
// CPI with PDA signing
const [pdaAddress, bump] = await PublicKey.findProgramAddress(
  [Buffer.from('authority')],
  programId
);

const instruction = new TransactionInstruction({
  keys: [
    { pubkey: pdaAddress, isSigner: false, isWritable: true },
    // ... other accounts
  ],
  programId: targetProgramId,
  data: instructionData
});

// Execute with PDA authority
await cpiService.executeCpi({
  callerProgramId: programId,
  targetProgramId: targetProgramId,
  instruction,
  accounts: instruction.keys,
```

Key Takeaways

CPI Architecture Benefits

- **Program Composability:** Build complex protocols from simple programs
- **Atomic Execution:** All CPI calls succeed or fail together
- **Security Isolation:** Each program maintains its own security model
- **Performance Efficiency:** No additional transaction overhead

SVM CPI Advantages

- **High Throughput:** Parallel CPI execution across programs
- **Shared State:** Programs can securely share and modify accounts
- **Gas Optimization:** Efficient resource usage for complex operations
- **Trustless Composition:** Programs can interact without trusted intermediaries